

Reviewer Pack — Noncommutative Geometry of Jordan Algebras Bounds DPLL Tree ...

Entry 2b0c5dcd6175 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 05:13:24 UTC

Noncommutative Geometry of Jordan Algebras Bounds DPLL Tree Height

Entry ID: 2b0c5dcd6175 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 05:13:24 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra (Jordan Algebras) **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

Statement:

For every Boolean formula F , the DPLL tree height $h(F)$ satisfies $h(F) \leq k * \log(n) + O(1)$, where n is the number of variables in F and k is a positive constant. Moreover, there exists a constructive mapping from each Jordan algebra associated with F to its corresponding DPLL tree that preserves this bound.

Rationale (proposer's reasoning):

Jordan algebras have been previously used to represent Boolean functions and can capture non-classical logics. A connection between the geometry of these algebras and the complexity of the DPLL algorithm could expose new structural insights into SAT solvers. This conjecture would bridge algebraic geometry with computational complexity, potentially leading to more efficient algorithms.

Taxonomy category: ACC_SIPSER (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 67fbda02d89df69e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

DPLL tree height $h(F)$ is within $O(1)$ of $k * \log(n)$ for all generated Boolean formulas F with $n \leq 40$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Jordan Algebras' AND 'DPLL tree complexity' - 'Noncommutative Geometry' AND 'DPLL tree height bound' - 'Mapping between Jordan algebras and DPLL trees'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_boolean_formula(n):
    if n == 1:
        return 'x'
    else:
        op = random.choice(['&', '|', '^'])
        left = generate_boolean_formula(random.randint(1, n//2))
        right = generate_boolean_formula(n - len(left.split('&')) - len(left.split('|')) - len(left.split('^')))
        return f'({left} {op} {right})'

def dpll_tree_height(formula):
    if formula.isalpha():
        return 0
    else:
        left, op, right = formula[1:-1].split()
        return max(dpll_tree_height(left), dpll_tree_height(right)) + 1

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        formula = generate_boolean_formula(n)
        height = dpll_tree_height(formula)
        k = 1 / math.log(n) # Example value for k, adjust as needed
        expected_bound = k * math.log(n)

        if height > expected_bound + 1: # 0(1) tolerance
```

```

        conjecture_holds = False
        counterexample = f"Formula: {formula}, Height: {height}, Expected Bound: {expected_bound}"

    total_metric_value += height
    instances_tested += 1

    return {
        "metric_name": "DPLL Tree Height",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}, \"instances_tested\": {result['instances_tested']}, \"conjecture_holds\": {result['conjecture_holds']}, \"counterexample\": \"{result['counterexample']}\"}}")

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std={std_metric_value:.4f} support={support_fraction:.4f}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value:.4f} std={std_metric_value:.4f} support={support_fraction:.4f}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_815f41c5.py", line 69, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_815f41c5.py", line 43, in run_trial
    formula = generate_boolean_formula(n)
              ^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_815f41c5.py", line 23, in generate_boolean_formula
    left = generate_boolean_formula(random.randint(1, n//2))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_815f41c5.py", line 24, in generate_boolean_formula
    right = generate_boolean_formula(n - len(left.split('&')) - len(left.split('|')) - len(left.split('^')))

```


3. The full LLM transcripts are in `research/audit/2b0c5dcd6175.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2b0c5dcd6175.tar.gz` (if generated)